



Grail

High School Information System — complete technical documentation of the front end, the back end, and the rules that connect them.

PROJECT

Grail HSIS v1.0

STACK

Laravel 12 · PHP 8.2+ · SQLite · Tailwind · Alpine

SCALE

40 tables · 31 models · 42 controllers · 155 views

TEAM

Group 40 — Copperbelt University

DOCUMENT DATE

14 September 2026

STATUS

11 of 16 phases complete

01 What the system is

Grail is a school information system for a Zambian secondary school. It replaces the paper and spreadsheet trail that runs between the office, the staff room, and the home — enrolment, marks, attendance, fees, timetables and report cards — and gives each of those four audiences its own portal onto the same data.

The central idea is that a school already has one set of facts; what it lacks is one place to keep them. A mark entered by a subject teacher should be the same mark the class teacher ranks, the same mark the parent reads on a report card, and the same mark the head teacher sees in a school-wide average. Grail is built so those four numbers cannot disagree, because they are all read from one row through one service.

4 ROLE PORTALS	40 DATABASE TABLES	155 SCREENS	11/16 PHASES COMPLETE
--------------------------	------------------------------	-----------------------	---------------------------------

The four portals

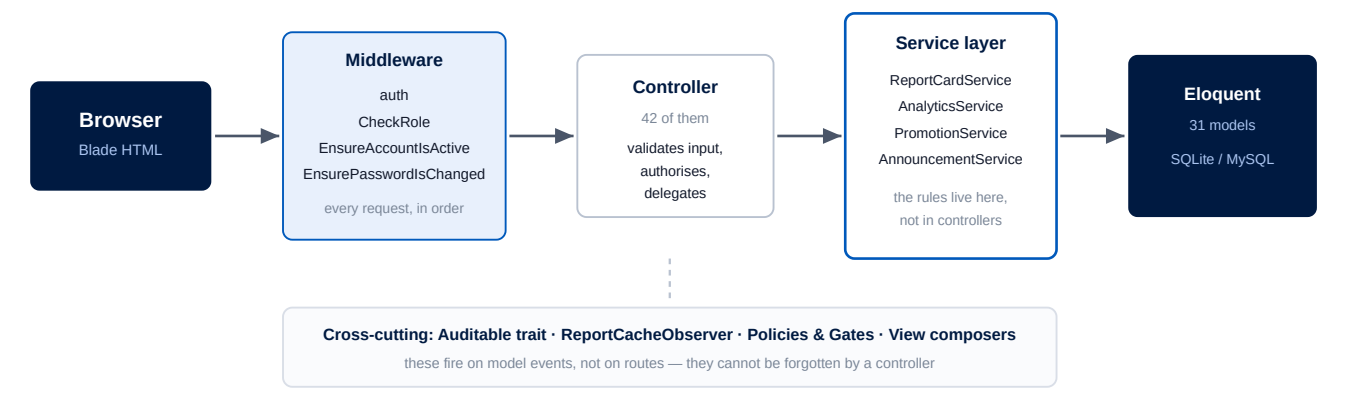
PORTAL	WHO	WHAT THEY CAN DO
Admin	Registrar, head teacher, bursar	Enrol students; manage teachers, parents, classes and subjects; build timetables; set and collect fees; run promotion at year end; publish announcements; read every report; manage logins and read the audit trail.
Teacher	Subject and class teachers	See their own classes and subjects; enter marks; record attendance; set and grade assignments; write report card comments; finalize the report cards of the class they are form teacher for.
Parent	Parents and guardians	See each of their children: results, attendance, timetable, assignments; fee balance with payment instructions and a reference code; download receipts and finalized report cards.
Student	Enrolled students	Their own dashboard, results, attendance, timetable, assignments with submission, announcements, and their finalized report cards.

Design principles the build actually follows

- **One source of truth per number.** Term averages are computed in exactly one place. The report card, the class performance screen and the school-wide report all call it.
- **Records are never silently destroyed.** Deletions are soft where history matters; a teaching assignment with marks against it refuses to be removed; a promotion can be rolled back.
- **Every consequential change is written down.** Creates, updates and deletes on the domain models append to an audit log with the actor, the field-level difference, and an optional reason.
- **The school's money never passes through the system.** Grail records what is owed and what was paid, and tells a parent how to pay the school directly. It processes no card and holds no funds.

02 Architecture

Grail is a server-rendered Laravel application. The browser receives finished HTML; JavaScript is a thin layer for interactions that genuinely need it. That choice keeps the system fast on the modest hardware and intermittent connections a Zambian school actually has, and it means a page still works when a script fails to load.



The path of a single request, and the machinery that runs regardless of which controller handles it.

Why a service layer exists at all

The first version of the plan put the averaging logic in each screen that needed it. That guarantees drift: the day someone changes how a CA mark is weighted on the report card and forgets the class performance page, the school has two official averages for the same child. Pulling those rules into services means a screen cannot compute its own answer — it has to ask.

SERVICE	OWNS	PUBLIC SURFACE
ReportCardService	Marks, averages, ranking, the finalize lock	build · subjectRows · averageOf · attendanceSummary · missingGrades · finalize · unfinalize · isLocked
AnalyticsService	Every report and its cache	attendanceReport · agingReport · agingBucket · studentAverages · passRate · classPerformance · schoolWideReport · remember · flush
PromotionService	Year-end rollover and its reversal	blockers · defaultFor · run · rollback · batches
AnnouncementService	Audience resolution and read state	audienceFor · reachSummary · reachFor · targetIds · markRead · markAllRead · unreadCount · feedFor

WORTH SAYING OUT LOUD IN THE PRESENTATION

AnalyticsService does not compute averages. It is constructed with ReportCardService and delegates to it. That is why the school-wide performance report and an individual child's report card are arithmetically incapable of disagreeing — there is only one implementation.

03 Technology stack

Back end

COMPONENT	VERSION
PHP	^8.2 (8.5 in dev)
Laravel	12.58
Database	SQLite (MySQL-portable)
ORM	Eloquent
Templating	Blade
PDF engine	barryvdh/laravel-dompdf ^3.1

Front end

COMPONENT	VERSION
Tailwind CSS	^3.1
Alpine.js	3.17
Vite	^6.0
Axios	^1.7
Icons	Material Symbols
Type	Inter · JetBrains Mono

Code inventory

50 MIGRATIONS	31 MODELS	42 CONTROLLERS	4 SERVICES
155 BLADE VIEWS	125 NAMED ROUTES	19 SEEDERS	25 TEST FILES

Why these choices

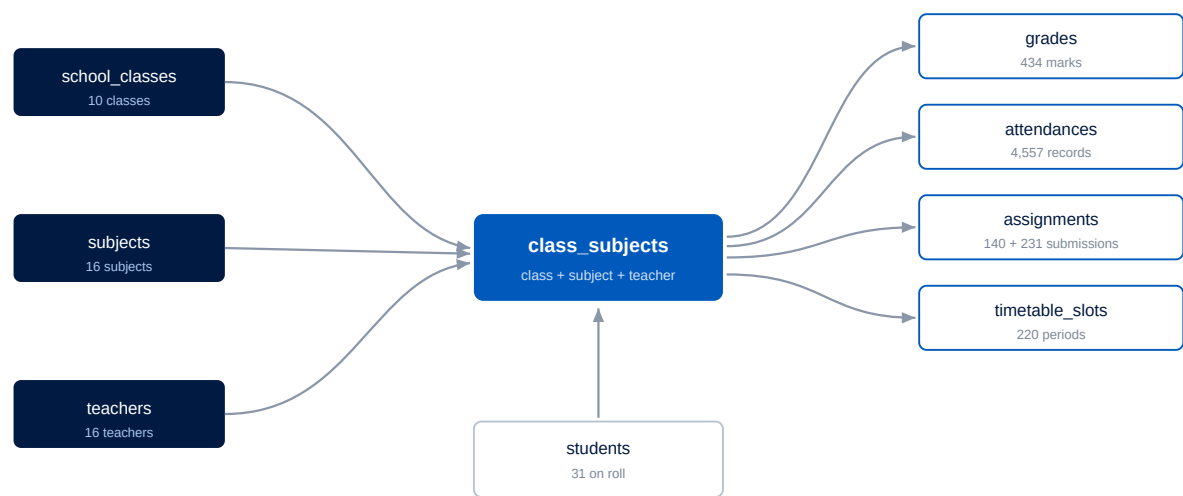
- **Server-rendered Blade over a SPA.** One deployable artifact, no API surface to secure separately, no client-side state to keep in step with the database. A school office on a slow connection gets a working page instead of a loading spinner.
- **Alpine over a framework.** The interactivity Grail needs is dropdowns, a retractable sidebar, a confirm dialog, a filter that submits itself. Alpine does that in fifteen kilobytes. Anything heavier would be paying a framework's price for a menu.
- **Tailwind with a custom token set.** Colours, type scale and spacing are defined once in `tailwind.config.js` as semantic names — `surface-container-lowest`, `on-surface-variant`, `sidebar-width` — so the four portals share one visual language without four stylesheets.
- **SQLite in development.** Zero setup for a five-person team. Every migration is written to run on MySQL too; the two that needed raw MySQL syntax are guarded by driver checks.

RECORDED DEVIATION

The original plan specified MySQL via XAMPP. The project moved to SQLite when XAMPP would not start, and four migrations were rewritten for portability. Both engines work; the plan's decision table records the change. Production verification against MySQL remains outstanding.

04 The data model

Forty tables, grouped into seven areas. Every domain table uses an explicit primary key named after its subject — `student_id`, `class_id`, `fee_id` — rather than a bare `id`, so a join reads like a sentence and a stray column cannot be silently mismatched.



A mark, an absence, an assignment and a timetable period all reference the same `class_subject` row.

`class_subjects` is the hinge of the whole schema: it says *who teaches what, to which class*, and everything academic hangs off it.

Tables by area

AREA	TABLES	WHAT IT ANSWERS
People & access	users · roles · students · teachers · parents	Who exists, who may sign in, and as what.
School structure	school_classes · subjects · class_subjects · grade_levels · periods	How the school is organised and who teaches where.
Calendar	academic_years · terms · holidays	Which term a mark or a fee belongs to, and how many school days it contained.
Academic record	grades · attendances · assignments · assignment_submissions · report_cards · report_card_comments	What each child did, and the formal statement of it.
Finance	fees · fee_items · fee_categories · payments · school_settings	What is owed, what was paid, and how to pay the school.
Communication	announcements · announcement_targets · announcement_reads	What was said, to whom, and who has read it.
Progression & oversight	promotion_mappings · student_promotions · audit_logs · timetable_slots	Movement between years, and a record of every change.

Relationships worth understanding

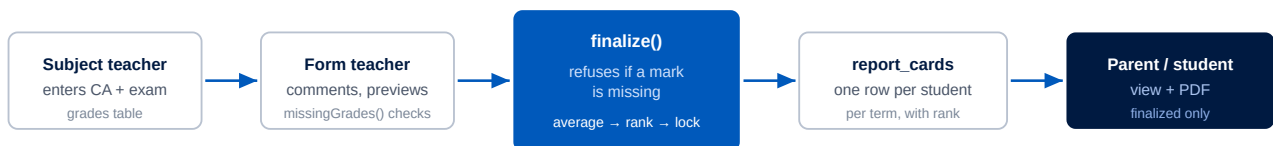
- **A teacher relates to a class in two different ways.** `school_classes.teacher_id` makes them the *form teacher* — pastoral responsibility, and the only person who may finalize that class's report cards.
`class_subjects.teacher_id` makes them a *subject teacher*. Conflating the two is the most common mistake a reader makes.
- **`class_subjects` is unique on (class, subject).** A subject is taught once per class. Reassigning it is therefore a hand-over, not a duplicate — and any marks already recorded move with it.
- **A fee is a header with line items.** `fees` carries the balance and status; `fee_items` carries tuition, boarding, uniform. Payments attach to the fee, never to an item.
- **Announcement targeting is a pivot, not a column.** One announcement can address several classes and several grade levels at once, so the targets live in `announcement_targets` as polymorphic rows.

05 The rules that carry weight

Five pieces of logic where getting it wrong would be visible to a parent. These are the parts worth demonstrating.

5.1 Report cards: averaging, ranking and the lock

A report card is not generated on demand from live marks — it is **finalized**, and finalizing does three things at once: it computes each student's term average, assigns ranks, and locks the class's marks for that term so nobody edits a grade after the cards have gone home.



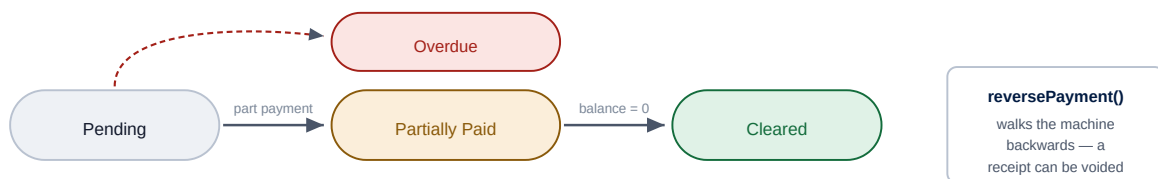
An unfinalized term is invisible to parents and students — a draft mark is not a result.

The report card pipeline. The gate in the middle is the point of the design.

- **Competition ranking.** Two students tied at second are both 2nd, and the next is 4th. Schools rank this way; a naive sort does not.
- **The finalize gate.** `missingGrades()` refuses the whole operation if any student is missing any subject, rather than producing cards with silent gaps.
- **Unfinalize requires a reason.** The reason is written to the audit log, and the cards disappear from the parent and student portals until re-finalized.
- **Attendance is collapsed to days.** A student attending seven subjects on Monday was present on *one* day, not seven. The report card counts calendar days, and the term's school-day total already excludes weekends and holidays.

5.2 Fees: a state machine, not a status field

Nothing writes `fees.status` directly. The balance is derived from payments, and the status is recomputed from the balance and the due date.



Overdue is applied by a scheduled command once the due date passes with a balance outstanding.

Fee statuses are computed, never assigned. That is what keeps a balance and its label in agreement.

5.3 Payment references the bursar can actually read

Grail takes no money. Instead a parent is shown the school's bank and mobile-money details, the exact amount owed, and a reference code to quote — GRL-0042-0117-A. The final character is a check character computed by weighted modulus over a 26-symbol alphabet with I, O, S, Z and B removed, because those are the characters people misread on a deposit slip. A mistyped or transposed reference fails to resolve rather than crediting the wrong child.

The design consequence is the important part: **the school's financial liability is unchanged by installing Grail**, because no funds ever pass through it.

5.4 Promotion: the year-end operation, and its undo

At year end the admin promotes each class, retains named individuals, and graduates the final year. The operation runs in a transaction, writes a `student_promotions` row per student recording where they came from and where they went, and stamps the batch with a reference.

- `blockers()` refuses to start when the target academic year is missing or report cards for the outgoing year are unfinalized.
- `rollback()` reverses a whole batch by its reference, because the realistic failure is an admin promoting the wrong class at 4pm on the last day of term.
- Classes are **not** duplicated per year. 8A is a permanent room; the year context lives on the promotion record. Duplicating classes would fork every timetable, fee and enrolment.

5.5 Announcements: one message, several audiences

An announcement carries a publish window and an audience. Audience resolution happens in one query that ORs the three cases — everyone, specific classes, specific grade levels — rather than looping. Read state is per user, so the unread badge in each portal is that user's own count. A student resolves to their own class; a parent resolves to the union of their children's classes.

06 Access control and accountability

Four roles, three middleware layers, two policies, and an audit log that nothing can route around.

The middleware chain

LAYER	WHAT IT ENFORCES
auth	Signed in at all.
CheckRole	This portal belongs to this role. A parent cannot reach a teacher URL by typing it.
EnsureAccountIsActive	A deactivated account is ejected on its next request, not merely blocked at the login form. Without this, deactivating someone already signed in would do nothing until they logged out.
EnsurePasswordIsChanged	An account created by an admin with a generated password is confined to its own settings page until it sets a real one.

Authorisation beyond the role

Role alone is too coarse for records. Two further mechanisms apply:

- **Ownership checks in the query.** A parent's child is resolved *through* the parent — a tampered student id in the URL returns 403, not another family's child. A teacher's class list is built from their own `class_subjects` rows, so there is no list to tamper with.
- **Policies and gates** — `viewReports`, `exportReports`, `viewFinancials` — guard the reporting and finance surfaces, so a role that may see academic reports does not automatically see the school's money.

Guards against locking the school out

The account management screen refuses three things: you cannot deactivate yourself, you cannot demote or deactivate the last remaining active administrator, and you cannot delete a person who still owns records. A system that lets an admin lock everyone out of the admin portal at 5pm on a Friday is not secure, it is merely strict.

The audit trail

The `Auditable` trait hooks model events, so a create, update or delete is recorded by the model itself rather than by whichever controller happened to perform it — there is no code path that can forget. Each entry stores the actor, the model, the action, a field-by-field difference, and an optional reason. The seeded database carries **1,260** audit entries.

DEMONSTRABLE IN THE PRESENTATION

Unfinalize a class's report cards with a reason, then open Audit Logs. The entry names the administrator, the class, the term, and the reason they typed. That is the accountability story in one click.

07 The front end

155 Blade views across four portals, sharing one token-based design system. Each portal has its own layout and navigation, but not its own design language.

PORTAL	VIEWS	SHELL
Admin	67	Fixed navy sidebar, grouped into collapsible sections; sticky header with student search.
Teacher	21	Light sidebar scoped to the teacher's own classes and subjects.
Student	15	Retractable sidebar — full width or a 76px icon rail, remembered per browser; an off-canvas drawer below 768px.
Parent	14	Child-switching shell; every screen is scoped to the selected child.
Shared	13	6 layouts, 6 auth screens, 1 report card template serving both screen and PDF.

The design system

Colour, type and spacing are semantic tokens in `tailwind.config.js`, not raw values in markup. A surface is `surface-container-lowest`; text on it is `on-surface-variant`; the sidebar is `sidebar-width` or `sidebar-rail`. Renaming a colour changes it everywhere at once, and no screen can quietly invent its own grey.

Where JavaScript is used, and where it deliberately is not

- **Alpine** drives the retractable sidebar, dropdowns, confirm dialogs and the copy-to-clipboard on a generated password.
- **Native `<details>`** drives the admin navigation groups — no JavaScript at all, and the group containing the current page is rendered open by the server.
- **Plain GET forms** drive every filter and search. Filters end up in the query string, so a filtered list can be bookmarked, shared and paginated without losing its filters.
- **No client-side state store.** The server is the state.

A LESSON THE BUILD LEARNED THE HARD WAY

The student sidebar originally hid its labels with `x-cloak` and took its width from an Alpine binding. When the asset bundle failed to load, the result was a sidebar stuck collapsed with no way to expand it — the control that would fix it was the thing that had disappeared. It now carries a zero-specificity CSS fallback, so with no JavaScript at all it renders fully expanded and usable; Alpine's classes override it the instant they load.

Navigation should never be the thing that breaks when a script fails.

Report card rendering

One Blade template serves both the on-screen preview and the dompdf download, switched by a single flag. The layout uses tables rather than flexbox because dompdf's CSS support stops short of modern layout, and the print toolbar is excluded from the PDF branch. A4 portrait, named after the student and term.

08 End to end: a mark becomes a report card

The single most useful thing to walk an audience through, because it touches every layer.

#	ACTOR	ACTION	WHAT HAPPENS UNDERNEATH
1	Mathematics Teacher	Opens Enter Marks, picks 8A · Mathematics, Term 1	The class-subject list is built from their own <code>class_subjects</code> rows — they cannot select a class they do not teach.
2	Mathematics Teacher	Enters CA and exam marks, saves	Rows written to grades against <code>class_subject_id</code> and <code>term_id</code> . The <code>Auditable</code> trait logs each one; <code>ReportCacheObserver</code> bumps the report cache version, so every cached report recomputes on next view.
3	Subject teachers	Write per-subject comments	Stored in <code>report_card_comments</code> , one per student per class-subject.
4	English Teacher (form teacher of 8A)	Opens Report Cards → 8A, writes overall comments, previews	<code>ReportCardService::build()</code> assembles subjects, attendance and term average live. No rank yet — it does not exist before finalizing.
5	English Teacher	Clicks Finalize	<code>missingGrades()</code> checks every student against every subject. If any are missing the operation is refused entirely. Otherwise averages are computed, competition ranks assigned, and one <code>report_cards</code> row written per student.
6	System	Locks the term	<code>isLocked()</code> now returns true for that class and term; the Enter Marks screen becomes read-only.
7	Parent	Opens Reports → their child	The card appears — resolved through the parent's own children, and only if finalized.
8	Parent	Downloads the PDF	<code>dompdf</code> renders the shared template with the print toolbar suppressed, A4 portrait.
9	Admin	Unfinalizes with a reason	Cards vanish from the parent and student portals, marks unlock, and the reason is written to the audit log.

Demo accounts

Every seeded account uses the password 12345678. Teachers are named after the subject they teach, so the staff room is readable and survives a re-seed.

ROLE	EMAIL	NOTES
Administrator	admin@grail.school	The only admin account.
Form teacher of 8A	english.language@grail.school	Can finalize 8A's report cards.
Subject teacher	mathematics@grail.school	Teaches Mathematics in all ten classes; form teacher of 8B.
Parent	parent@grail.school	Parent of Demo Student, 8A.
Student	student@grail.school	Demo Student, 8A.

Other subject teachers follow the same pattern: `geography@grail.school`, `civic.education@grail.school`, `integrated.science@grail.school`, and so on for all sixteen subjects.

09 Status against the plan

11 PHASES COMPLETE	1 PARTIALLY DONE	3 NOT STARTED	1 NEEDS A DECISION
------------------------------	----------------------------	-------------------------	------------------------------

PHASE	SCOPE	STATE	NOTE
00	Environment & database	DEVIATED	SQLite instead of MySQL/XAMPP. Recorded in the plan.
01	Fee status correctness	COMPLETE	State machine replaced a literal string.
02	Multi-item fees & audit trail	COMPLETE	fee_items, audit_logs, Auditable trait, log viewer.
03	School calendar	COMPLETE	Years, terms, holidays, grade levels, school-day counting.
04	Role dashboards	COMPLETE	All four portals real; no placeholder route remains.
05	Announcements with targeting	COMPLETE	Polymorphic targets, read tracking, unread badges.
06	Promotion & year-end rollover	COMPLETE	Pre-flight blockers, audit trail, reversible batches.
07	Receipts & overdue notices	COMPLETE	Needs one cron line to self-run.
08	Spec doc: Bootstrap → Tailwind	NOT STARTED	Documentation only; roughly half a day.
09	Reporting suite	COMPLETE	Attendance, aging, class performance, school-wide; all with CSV.
10	Class timetable	COMPLETE	Per-grade periods, overlap and double-booking guards.
11	Report cards & class rank	COMPLETE	Finalize workflow, tie-aware ranking, PDF across portals.
12	Account management	COMPLETE	Deactivation, temporary passwords, audit log diffs.
13	Business-logic tests	PARTIAL	25 test files; the newest features are uncovered.
14	Mobile responsiveness	NOT STARTED	Student portal is responsive; the other three are not.
15	Production alignment & UAT	NOT STARTED	Backups, retention, rate limiting, user acceptance testing.

Built, though never in the plan

- **Assignments** — two tables, teacher authoring with draft/publish, student submission with file upload, marking with a "not handed in" list, and parent visibility.
- **Payment instructions and reference codes** — bank and mobile money details configurable by the admin, shown to parents with the exact amount owed and a typo-resistant reference.

10 Honest limitations

Every one of these is known, deliberate or scheduled. Saying them before an examiner finds them is the stronger position.

LIMITATION	DETAIL, AND WHAT IT WOULD TAKE
Test coverage lags the build	25 test files cover auth, admin pages, the parent portal, marks entry and timetables. Assignments, report cards, ranking, announcements, promotion and the reporting suite have none. The logic in each was proven with standalone tests during development; converting those into PHPUnit is the outstanding work, and it is Phase 13.
Only one student has a login	31 student records exist; the seeder creates one student account. Fine for a demo, but a second student cannot sign in without one being created.
Mobile pass incomplete	The student portal was built responsive from the start. Admin, teacher and parent have had no narrow-viewport pass.
Timetable is nominally over-committed	With one teacher per subject school-wide, a teacher is timetabled in several grades at the same clock time. The clash guard compares period rows, not times, so it does not object. Realistic fixes: more than one teacher per subject, or tighten the guard to compare start times.
Two legacy columns remain	<code>users.role</code> (a string, superseded by <code>role_id</code>) and the now-unused <code>teacher_subjects</code> table. Both are read by almost nothing and have each caused a bug; dropping them is a small migration.
Email is not configured	The mail driver writes to a log, and seeded addresses are not real. This is why admin-issued passwords are shown on screen rather than emailed — which also matches how a school office actually hands over a password.
No payment gateway	Deliberate. Grail records obligations and receipts; the school is paid directly. Integrating a gateway would place the school's funds and PCI obligations inside a student project.
Production not yet verified	Backup policy, retention, rate limiting, session timeout and structured user-acceptance testing are Phase 15 and not begun. The application has been exercised in a browser; it has not been run under load or against MySQL.

Verification performed

All 27 admin screens were loaded in a browser and checked for server errors — all returned 200. The student search and filters were exercised across fifteen query combinations against live data. The CSV export was verified to honour active filters. The report card, announcement, promotion, ranking and payment-reference algorithms were each proven with standalone logic tests before integration.

IF ASKED "WHAT WOULD YOU DO NEXT?"

Tests, in one word. Every defect found late in this build — a broken relationship name, a missing trait, an unguarded migration, conflict markers committed to the main branch — is one a test would have caught the same hour it was introduced. That is the honest answer, and it is a better one than claiming the system is finished.

11 Reference

Glossary for the audience

TERM	MEANING IN GRAIL
Class subject	The pairing of one subject, one class and one teacher. The unit everything academic references.
Form teacher	The teacher responsible for a class as a whole. The only person who may finalize its report cards.
Finalize	Turning a term's live marks into formal report cards: computing averages, assigning ranks, and locking the marks.
Competition ranking	Ties share a position and the next position is skipped — 1, 1, 3 rather than 1, 1, 2.
Aging bucket	How overdue a fee is, grouped as not yet due, 1–30, 31–60, 61–90, or 90+ days.
Payment reference	The code a parent quotes when paying the school, carrying a check character so a misread cannot credit the wrong child.
Promotion batch	One year-end rollover, reversible as a unit by its reference.
Audit log	The record of who changed what, when, and why.

Seeded dataset

User accounts	49	Marks recorded	434
Students	31	Attendance records	4,557
Teachers	16	Assignments	140
Parents	31	Submissions	231
Classes	10	Timetable periods	220
Subjects	16	Fees raised	31
Class subjects	70	Audit entries	1,260

Running it

COMMAND	PURPOSE
<code>composer install</code>	PHP dependencies.
<code>npm install && npm run build</code>	Compile CSS and JavaScript into public/build.
<code>php artisan migrate:fresh --seed</code>	Build the database and populate the demo school.
<code>php artisan serve</code>	Serve at 127.0.0.1:8000.
<code>php artisan optimize:clear</code>	Clear compiled views and config after changes.

ONE DEPLOYMENT NOTE

If the application ever renders as unstyled HTML, the cause is almost always a leftover `public/hot` file. Laravel checks for it before the compiled manifest, and if present loads every asset from the development server URL inside it — which fails once that server stops. Deleting the file restores the built assets.